Data Structures and Algorithms — Lab 1

Topic

- Introduction to GitHub and version control.
- Introduction to Sorting and Searching Algorithms using Templates.

Objective

- Understand the fundamentals of version control and collaboration using GitHub.
- Understand and implement the Selection Sort algorithm.
- Explore the concepts of Linear Search and Binary Search algorithms.
- Develop code using templates to make the sorting and searching algorithms reusable for different data types.

Outcomes

1. Understand the basics of version control by creating and managing repositories on GitHub.
2. Implement Selection Sort using templates to organize arrays of various data types in ascending order.
3. Use Linear Search with templates to find elements in unsorted arrays of different data types.
4. Apply Binary Search using templates on sorted arrays to improve search efficiency for multiple data types.

Content

The following sections will be covered during this lab session.

1. **GitHub** will cover creating accounts and private repositories, uploading code files, managing versions, making changes and committing updates through the web interface, viewing commit history, reverting to previous versions, and uploading Scratch projects for version control.
2. Commit, track, and revert code changes on GitHub.
3. **Selection Sort** will cover implementing the algorithm by repeatedly finding the smallest element in the unsorted part of the array and swapping it with the current element. Students will write, compile, and execute the code, ensuring the program sorts the array correctly.
4. **Linear Search** will involve writing code to search for a specific value by iterating through each element in the array sequentially. Students will compile and execute the code, testing it with different inputs to ensure correctness.
5. **Binary Search** will guide students through implementing the algorithm on sorted arrays, using an iterative method to divide the search space by half at each step. Students will write, compile, and test the code with edge cases such as single-element arrays or missing values.

Task Instructions

Students are required to complete the following tasks in lab timings.

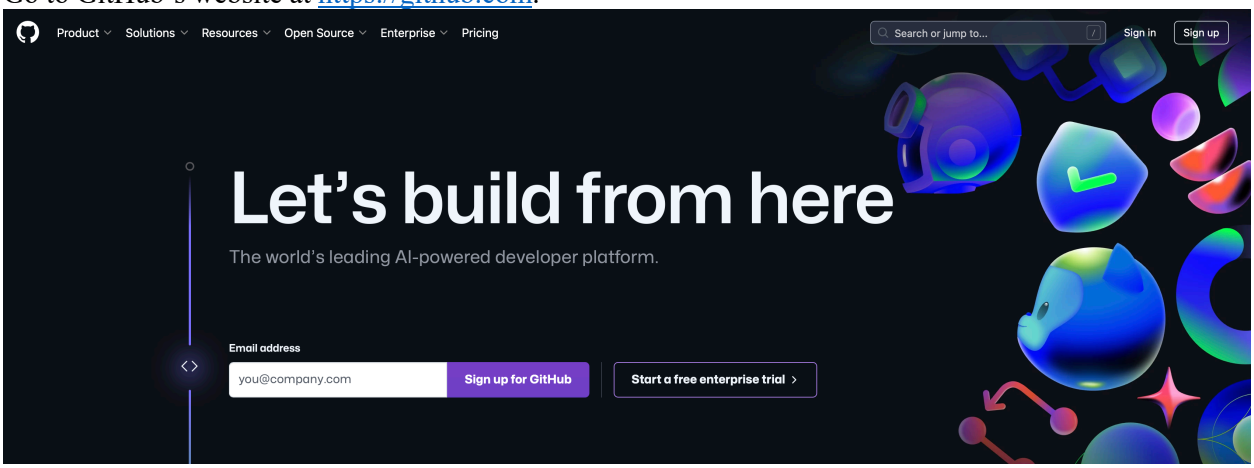
Task – Introduction to GitHub

GitHub is a web-based platform that allows you to store, manage, and collaborate on software projects. It uses **Git**, a version control system, to track changes to your code over time. GitHub enables you to easily share your code with others, collaborate on projects, and maintain different versions of your work. Whether you are working solo or with a team, GitHub helps ensure that you have a full history of your project, can revert to earlier versions if needed, and efficiently collaborate by merging changes from different contributors.

We use GitHub because it provides a centralized place to store your code, track progress, and collaborate with others. It also integrates with many tools and services, making it easier to work on coding projects from anywhere. In this guide, we will cover how to create an account on GitHub, create your first repository, upload a simple C++ file, make changes to the file and commit the new version, view different versions, and delete a repository when needed.

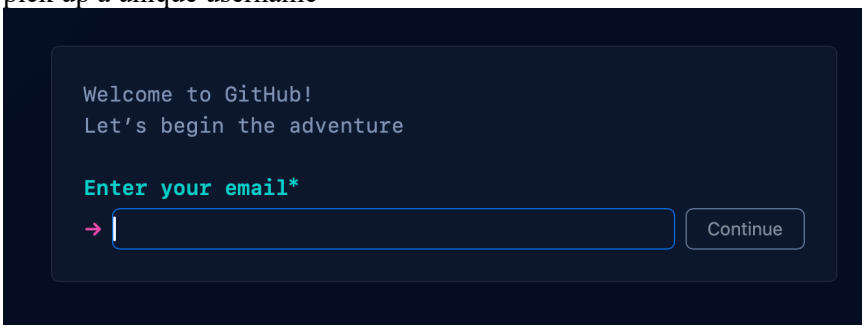
1. Creating an Account on GitHub

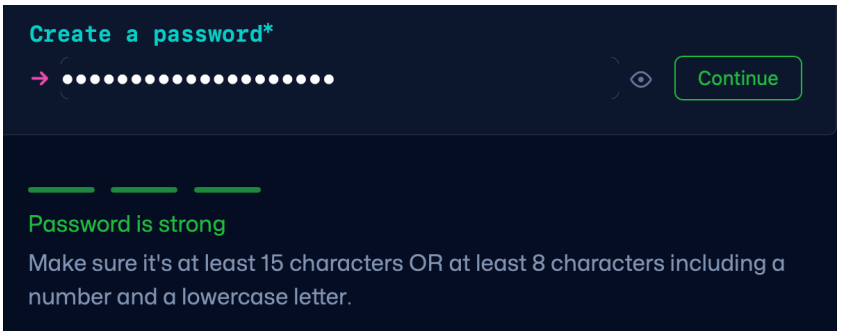
Go to GitHub's website at https://github.com.

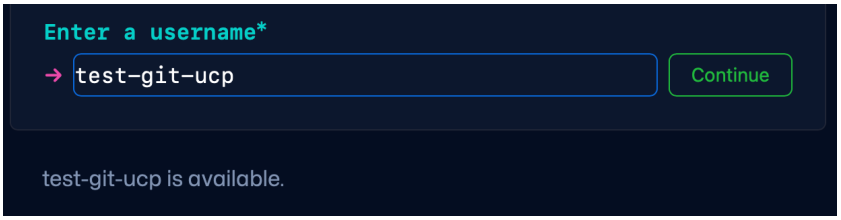


Click on the "Sign up" button in the upper-right corner of the homepage.

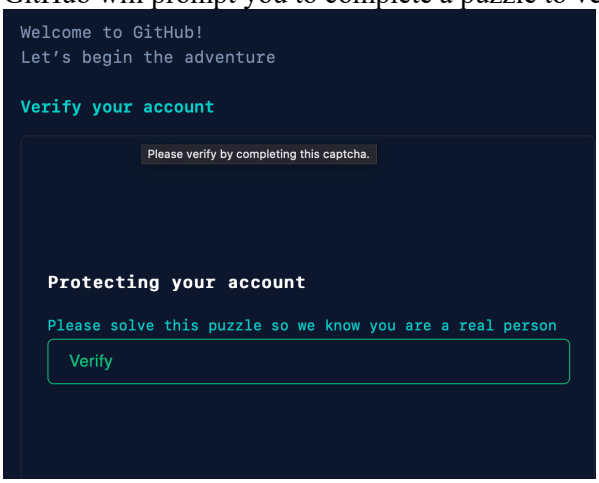
Enter your details: choose a unique username, enter your email address, and create a strong password and pick up a unique username

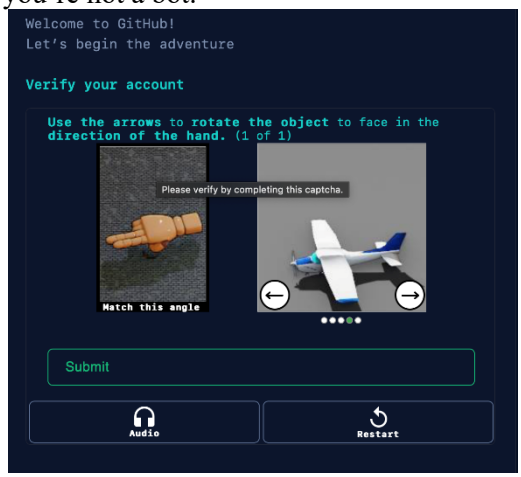




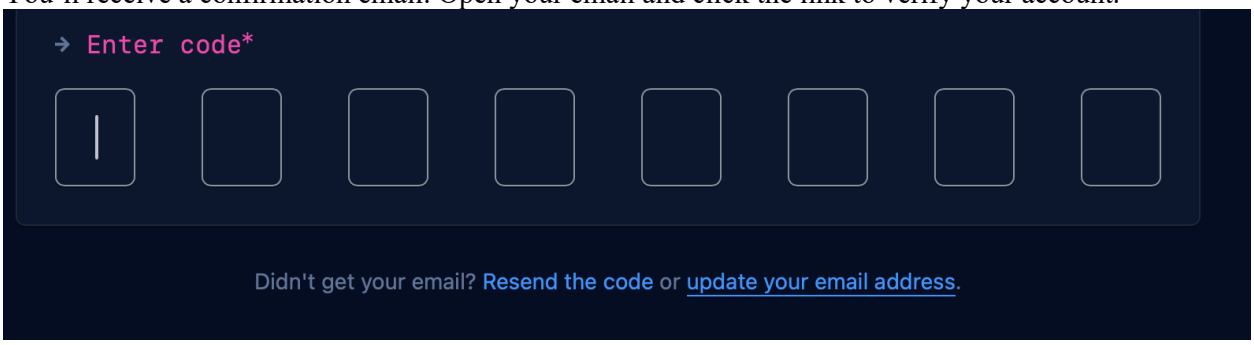


GitHub will prompt you to complete a puzzle to verify you're not a bot.





You'll receive a confirmation email. Open your email and click the link to verify your account.

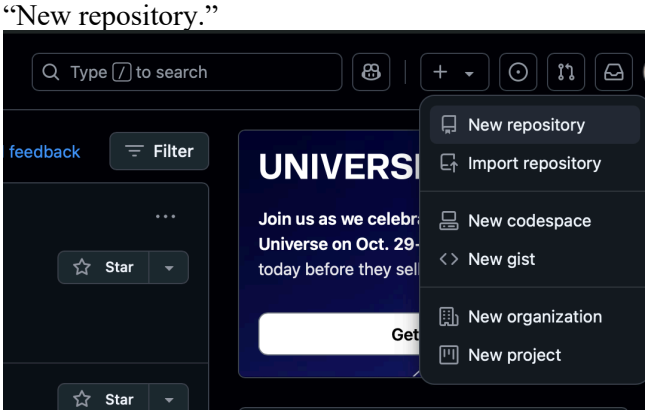


If asked, pick a plan: for most beginners, the free plan is sufficient. You'll then be taken to your GitHub dashboard, where you can customize your settings, but you can skip this step if preferred.

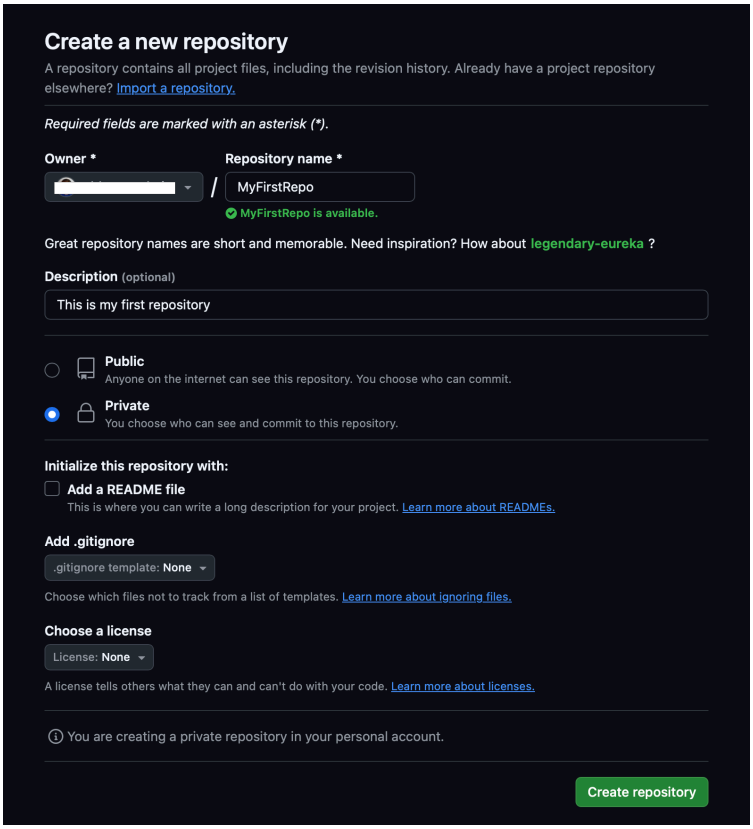
2. Creating Your First Repository and Uploading myFirstProgram.cpp

Step 1: Create a Private Repository

Go to the dashboard. After logging in, click the plus icon in the upper-right corner of the page and select "New repository."

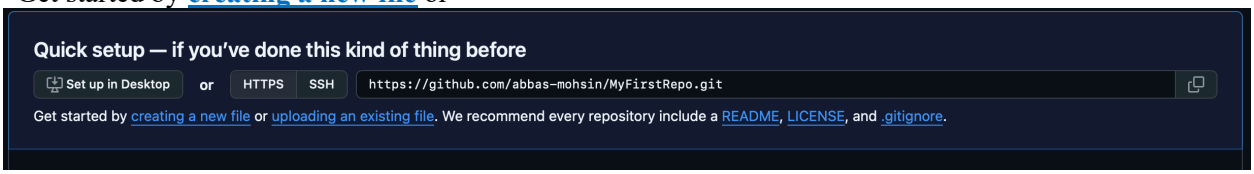


Enter a name for your repository (for example, "MyFirstRepo"). Optionally, add a short description of the repository. Select the "Private" option to ensure the repository is private and only accessible to you (and invited collaborators). Optionally, you can check the option to "Add a README file," but leave other options unchecked for now. Click "Create repository" at the bottom of the page.

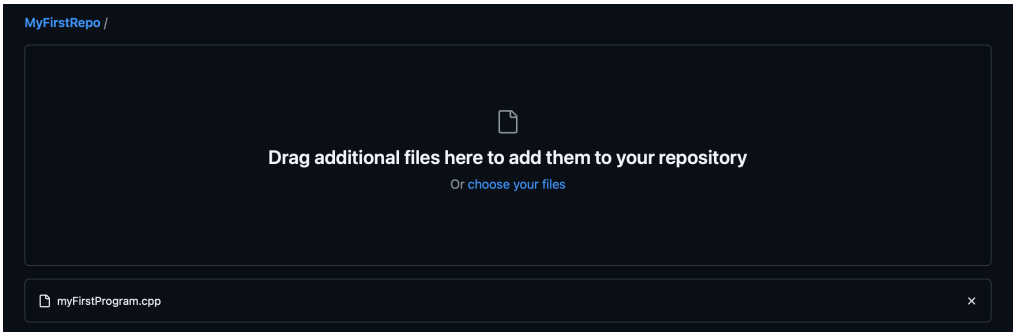


Step 2: Upload myFirstProgram.cpp

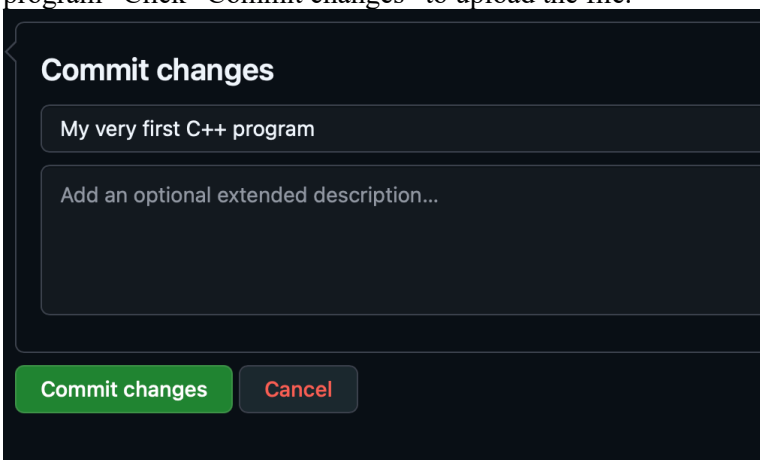
Once you're on your repository's main page, click "uploading an existing file" – you will find it right after: "Get started by **creating a new file** or "

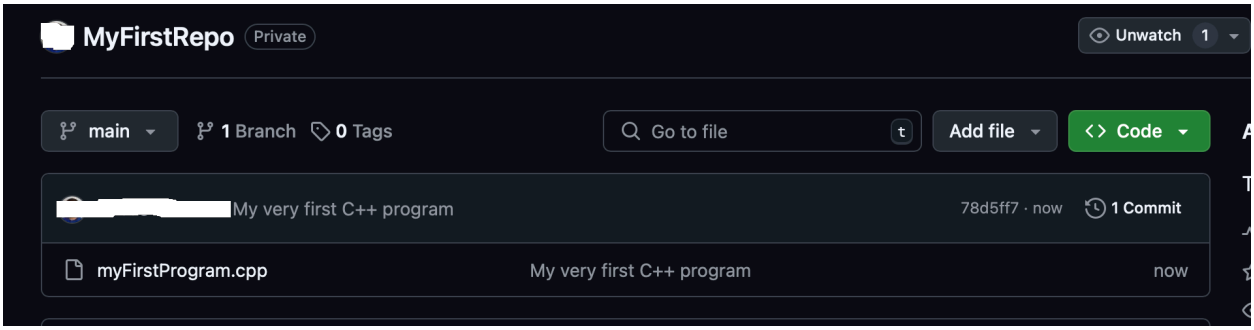


Drag your **myFirstProgram.cpp** file from your computer into the designated area or click "choose your files" to browse and select it.



In the "Commit changes" section, write a message describing the upload, for example, "My very first C++ program" Click "Commit changes" to upload the file.

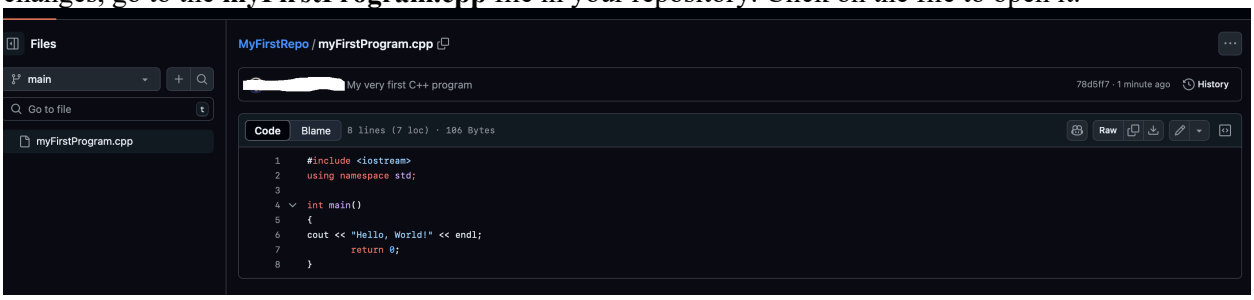




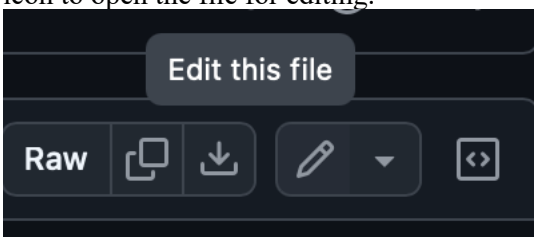
Your **myFirstProgram.cpp** file is now uploaded to your private GitHub repository.

3. Making Changes and Committing Them

Once you've uploaded your code, you can easily make changes to it through GitHub's website. To make changes, go to the **myFirstProgram.cpp** file in your repository. Click on the file to open it.

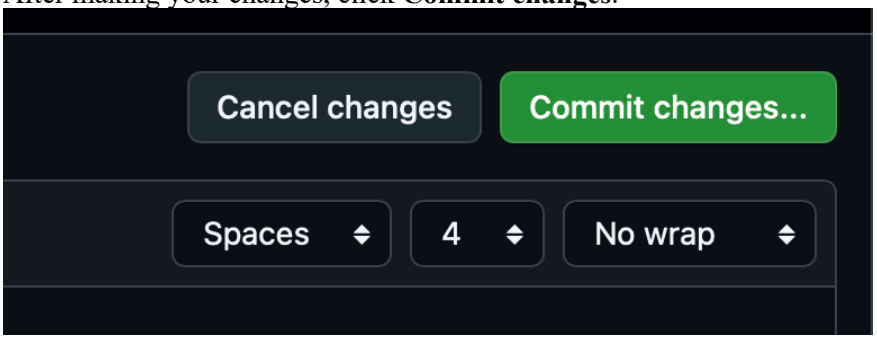


At the top-right corner of the file, you'll see a pencil icon that says **"Edit this file"**. Click on this pencil icon to open the file for editing.

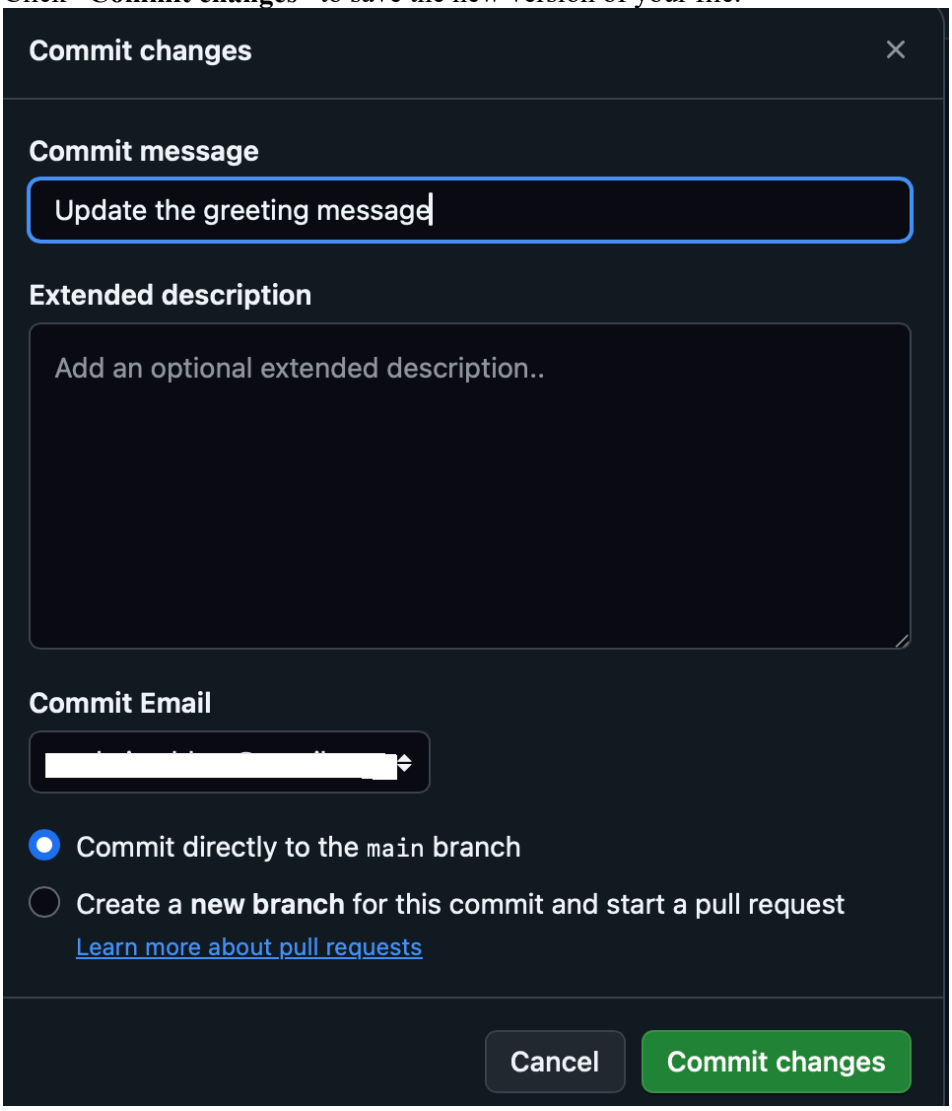


You can now make changes to your code. For example, change the output of your program from `Hello, World!` to something else, like `Hello, GitHub!`

After making your changes, click **Commit changes**.



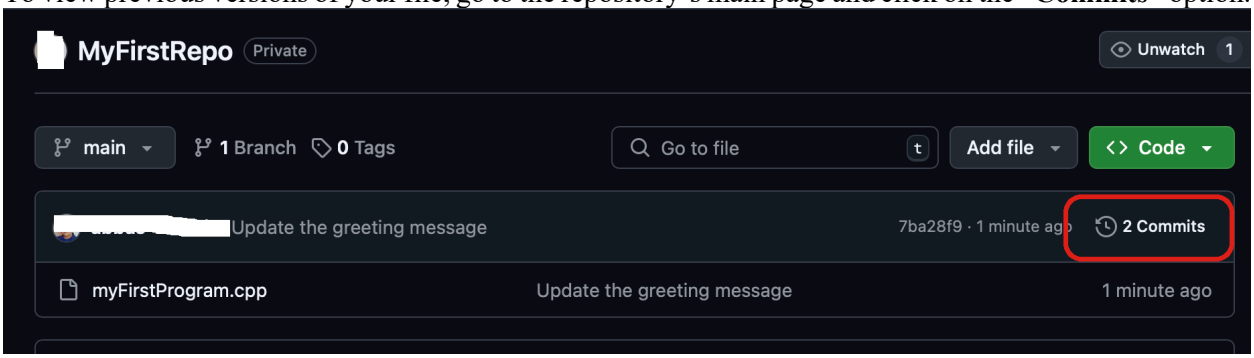
In the **Commit message** field, describe your changes (for example, "Update the greeting message").
You can leave the default option **"Commit directly to the main branch"** selected.
Click **"Commit changes"** to save the new version of your file.



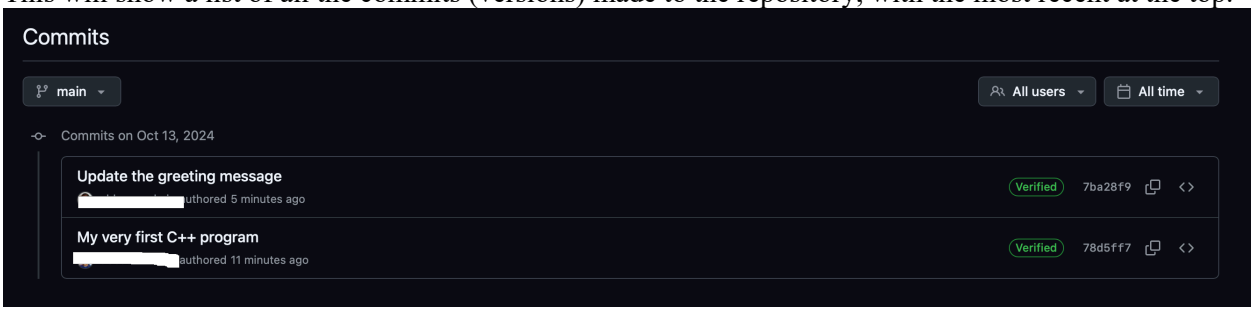
4. Viewing Different Versions of Your File

Both the original and the updated versions of your file will be saved as part of your repository's history. GitHub tracks these changes as different "versions" of your file, allowing you to revert back to an older version if needed.

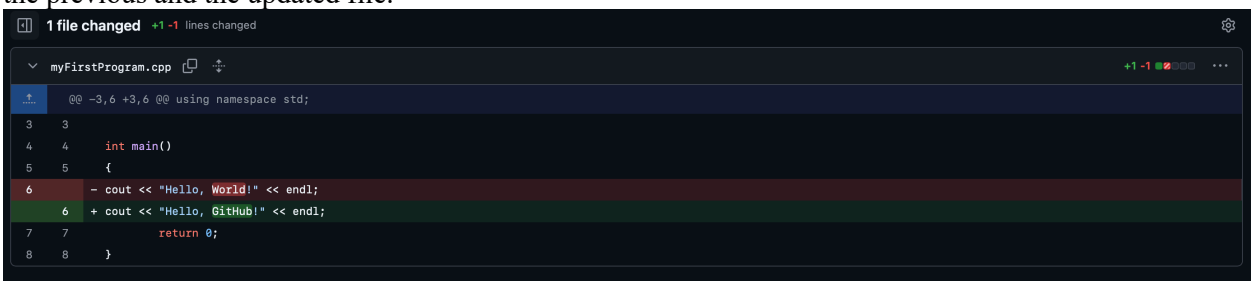
To view previous versions of your file, go to the repository's main page and click on the **"Commits"** option.



This will show a list of all the commits (versions) made to the repository, with the most recent at the top.



Click on any of the commits to see the changes made in that version. You can view the differences between the previous and the updated file.



This way, you can always go back to an earlier version of the file if needed, ensuring you never lose important work.

Task 1

Selection sort is a sorting algorithm in which we repeatedly find the next largest (or smallest) element in the array and move it to its final position in the sorted array. Assume that we wish to sort the array in increasing (ascending) order, i.e., the smallest element at the beginning of the array and the largest element at the end. We begin by selecting the smallest element and moving it to the lowest index position. We can do this by swapping the element at the lowest index and the smallest element. We then reduce the effective size of the array by one element and repeat the process on the greater (sub)array. The process stops when the effective size of the array becomes 1 (an array of 1 element is already sorted). The pseudocode of the task is given below:

```
Input: An unsorted array, A of N elements
Output: Sorted array, A

For I = 0:N-1
    SmallSub = I
    For J = I+1:N-1
       If A[J] < A[SmallSub]
          SmallSub = J
       End-If
    End-For
    Swap ( A[I], A[SmallSub] )
End-For
```

Implement the code for selection sort using templates. Test your code using the main function below and upload it to your repository: **Sort_Search_DSA.** The name of the code on your repository should be **Task_1.cpp**

```
int main() {
    // Test with an integer array of size 5
    int intArray[5] = {64, 25, 12, 22, 11};
    cout << "Original integer array: ";
    printArray(intArray);
    selectionSort(intArray);
    cout << "Sorted integer array: ";
    printArray(intArray);

    // Test with a string array of size 4
    string stringArray[4] = {"apple", "orange", "banana", "grape"};
    cout << "\nOriginal string array: ";
    printArray(stringArray);
    selectionSort(stringArray);
    cout << "Sorted string array: ";
    printArray(stringArray);

    return 0;
}
```

Task 2

In computer science, linear search or sequential search is a method for finding a particular value in a list that consists of checking every one of its elements, one at a time and in sequence, until the desired one is found. The pseudocode of the task is given below:

```
For I = 0:N-1
```

```
   If ( A[I] == Value )
     Return [I];
   End-if
End-For
return -1;
```

Implement the code for linear sort using templates. Test your code using the main function below and upload it to your repository: **Sort_Search_DSA.** The name of the code on your repository should be **Task_2.cpp**

```
int main() {
    // Test with an integer array of size 5
    int intArray[5] = {64, 25, 12, 22, 11};
    int intKey = 12;
    int intIndex = linearSearch(intArray, intKey);
    printSearchResult(intIndex, intKey);

    // Test with a float array of size 4
    float floatArray[4] = {3.14, 2.71, 1.62, 0.57};
    float floatKey = 1.62;
    int floatIndex = linearSearch(floatArray, floatKey);
    printSearchResult(floatIndex, floatKey);

    // Test with a string array of size 4
    string stringArray[4] = {"apple", "orange", "banana", "grape"};
    string stringKey = "banana";
    int stringIndex = linearSearch(stringArray, stringKey);
    printSearchResult(stringIndex, stringKey);

    return 0;
}
```

Task 3

Binary search is another searching algorithm used to search a specific value (or index of value) from the *sorted array*. In binary search, we first compare the *value to be searched* with the item in the middle position of the array. If there's a match, we can return immediately. If the key is less than the middle key, then the item sought must lie in the lower half of the array; if it's greater than the item sought must lie in the upper half of the array. So we repeat the procedure on the lower (or upper) half of the array. The pseudocode of the task is given below:

```
Input: A Sorted array, A of N elements and value to be searched
Output: Index of searched element or -1 if not found

low = 0, high = N-1;
While (low <= high)
   mid = (low + high)/2;
   If ( A[mid] == Value )
     Return mid;
   Else-if ( A[mid] < Value )
     low = mid + 1;
   Else
     high = mid - 1;
   End-if
End-While
return -1;
```

Implement the code for binary search using templates. Test your code using the main function below and upload it to your repository: **Sort_Search_DSA.** The name of the code on your repository should be **Task_3.cpp**

```
int main() {
    // Test with an integer array (sorted) of size 5
    int intArray[5] = {11, 12, 22, 25, 64};
    int intKey = 22;
    int intIndex = binarySearch(intArray, intKey);
    printSearchResult(intIndex, intKey);

    // Test with a float array (sorted) of size 4
    float floatArray[4] = {0.57, 1.62, 2.71, 3.14};
    float floatKey = 2.71;
    int floatIndex = binarySearch(floatArray, floatKey);
    printSearchResult(floatIndex, floatKey);

    // Test with a string array (sorted) of size 4
    string stringArray[4] = {"apple", "banana", "grape", "orange"};
    string stringKey = "grape";
    int stringIndex = binarySearch(stringArray, stringKey);
    printSearchResult(stringIndex, stringKey);

    return 0;
}
```